

Flow-Pattern Anomaly Detection at the IoT Gateway

A Pre-trained Mamba Approach

DRDO Internship Proposal

Madhav Dogra

May 24, 2026

Abstract

We propose an anomaly detector for the **smart-home IoT** traffic path (cameras, sensors, smart bulbs, voice assistants, and similar consumer devices) that is deployable at the home gateway, works on encrypted traffic, and detects both known attacks and zero-day flows. The detector is built on three architectural commitments that distinguish it from conventional IoT IDS work. **First**, each flow is represented as an ordered sequence of per-packet features — a *flow pattern* — rather than as an aggregated statistic vector. **Second**, a Mamba selective state-space-model encoder is pre-trained self-supervised on unlabelled benign traffic, producing a representation that is then shared by all downstream tasks. **Third**, the backbone is a state-space model rather than a transformer, on the grounds that recent work (NetMamba) has demonstrated comparable accuracy to BERT-class encrypted-traffic models such as ET-BERT at a fraction of the parameter and latency cost. Two downstream heads consume the encoder: a supervised softmax classifier for the seven CICIOT2023 attack categories and a one-class anomaly scorer for zero-day detection. The system is evaluated on CICIOT2023 (primary), TON_IoT (cross-dataset generalisation), and IoT-23 (real-world malware). SHAP attributions, an FGSM adversarial-robustness study, and a post-quantisation latency / memory benchmark on gateway-class hardware complete the deliverable. The 12-week plan culminates in a reproducible code repository, a pre-trained encoder checkpoint, a benchmark table, and a written report.

1 Background and Motivation

1.1 Why smart-home IoT is the soft layer of modern networks

The detector targets the class of devices typically found in a **smart home**: IP cameras, environmental and motion sensors, smart bulbs, voice assistants, thermostats, doorbells, smart plugs, and the home gateway / router itself. These devices are now the most exposed and least defended surface in most networks. They ship with weak default credentials, run firmware that is rarely patched, frequently have physical-access exposure, and lack the compute headroom for endpoint-resident security tooling. The Mirai botnet, built almost entirely from compromised IP cameras and home routers, is the canonical demonstration that this surface is exploitable at scale. CICIOT2023’s 105-device testbed is dominated by exactly this class of consumer IoT, which makes it the right training corpus.

1.2 Why detect at the gateway, not at the device

Endpoint-resident agents are not an option for most IoT devices, so detection must happen on the network path. Within that path, the IoT gateway is the natural choke point: every device flow passes through it on its way to the central collector, and the gateway is the only place where the operator can deploy modern compute without instrumenting every device. We

therefore target the gateway as the deployment environment. This commits us to a tight latency and memory budget — gateway hardware is typically Raspberry-Pi-class — but gives us full visibility over the upstream traffic.

1.3 Why flow patterns, not aggregated statistics

Conventional IoT IDS work aggregates each flow into a fixed-size vector of summary statistics — mean packet length, total bytes, IAT standard deviation, flag counts, and so on. This representation is convenient but lossy: it is permutation-invariant by construction, so the temporal structure of a flow is discarded. Yet many attack signatures *live* in the temporal structure: beaconing patterns, scan progressions, handshake oddities, burst behaviour. The alternative is to keep the flow as a sequence — the ordered per-packet feature vectors of (say) the first 32 packets of the connection — and let a sequence model do the aggregation implicitly through learning. We adopt this representation.

1.4 Why a state-space model, not a transformer

The reference architecture for pre-trained encrypted-traffic representations is ET-BERT [1], a BERT-base-scale masked-language-model trained on raw datagram bytes. ET-BERT achieves state-of-the-art accuracy on multiple encrypted-traffic classification benchmarks, but at ≈ 110 M parameters it is structurally incompatible with gateway-class hardware. The selective state-space model family (S4D [4], Mamba [5], Mamba-2 [6]) gives us a way around this constraint: linear-time inference in sequence length, transformer-comparable quality on long-sequence tasks, and substantially lower parameter counts at comparable accuracy. NetMamba [2] applies exactly this idea to network traffic classification, reporting ET-BERT-class accuracy at a fraction of the cost. We adopt the NetMamba design philosophy as the core of our detector.

1.5 Four pillars of the proposed design

The proposal sits on four explicit design commitments:

1. **Flow patterns**, not aggregated statistics, as the input representation.
2. **Self-supervised pre-training** of a shared encoder on benign-only traffic, before any supervised signal is applied.
3. **A Mamba backbone**, chosen for the cost-quality trade-off on long-sequence input at edge scale.
4. **Analyst-facing evaluation**: SHAP attributions on every alert, plus an FGSM adversarial stress test, on top of standard classification metrics.

2 Related Work and Architectural Choices

2.1 Pre-trained representations for network traffic

ET-BERT [1] (Lin et al., WWW 2022) is the reference architecture for the pre-train-then-fine-tune paradigm on network traffic. It tokenises raw datagram bytes, pre-trains a BERT-base transformer with masked-token and same-origin-sentence objectives, and fine-tunes on downstream classification. It defines the paradigm we follow. Its cost is the reason we do not use it directly.

NetMamba [2] (Wang et al., 2024) is the closest precedent to our work. It replaces ET-BERT’s transformer with a unidirectional Mamba, pre-trains on packet-byte sequences, and reports

accuracy competitive with ET-BERT at substantially lower parameter count and inference latency. We use NetMamba as the architectural template for the encoder.

ET-SSL [3] establishes that self-supervised contrastive learning over flow-statistic views of encrypted traffic produces representations transferable to anomaly detection. It motivates an optional contrastive variant of our pre-training objective.

2.2 The state-space-model line

S4D [4] (Gu, Goel, Ré, NeurIPS 2022) is the diagonal parameterisation of structured state-space models. It is the practical, easy-to-train form of S4 on which all subsequent work in this family builds. Foundational reading.

Mamba [5] (Gu & Dao, 2023) introduces *selective* state-space models: the state-transition matrix becomes input-dependent, which gives the model the capacity to selectively retain or discard information along the sequence. The result is linear-time inference in sequence length with transformer-comparable quality. Our encoder uses Mamba blocks.

Mamba-2 / Structured State Space Duality (SSD) [6] (Dao & Gu, ICML 2024) shows that transformers and SSMs are two views of a single underlying structure, yielding a faster and more expressive successor to Mamba. Reserved as a stretch-goal swap.

2.3 IoT IDS datasets, benchmarks, and surveys

The current default benchmark is **CICIoT2023** [7] (Neto et al., 2023), a real-device capture of 105 devices and 33 attack types across 7 categories. **TON_IoT** is widely used as a cross-dataset generalisation target. **IoT-23** (Stratosphere Lab) contains real-world malware captures including Mirai, Torii, and Okiru. Background context is set by the comprehensive review of Morshedi et al. [8] and the MDPI survey [9]. The transformer-cost-on-gateway argument is empirically grounded in the Nature *Sci. Rep.* study [10], which directly motivates our move to SSMs.

2.4 Explainability and adversarial robustness

We adopt SHAP [11] as the per-alert explanation mechanism — the analyst-facing standard for IDS literature. For adversarial evaluation we follow the recent IoT-IDS adversarial-robustness study [12], which uses FGSM as the entry-point benchmark and shows that SHAP DeepExplainer attribution fingerprints separate clean and adversarial inputs — a useful secondary signal.

2.5 Stretch-goal references

For the optional graph-aware extension we follow E-GraphSAGE [13] (Lo et al.). For federated training across DRDO sensor sites that cannot share raw pcaps we draw on the hybrid DL+FL IDS work [14] and PEIoT-DS [15].

3 Proposed System

3.1 At a glance

A flow arriving at the IoT gateway is reduced to a fixed-length sequence of per-packet feature vectors — the flow pattern. This sequence is encoded by a Mamba network whose weights have been pre-trained self-supervised on a large set of benign flows. The resulting embedding is read by two downstream heads in parallel: a supervised classifier for known attack categories and a one-class anomaly scorer for zero-day detection. A rule-based fusion gate raises an alert if either

head exceeds threshold; SHAP attributions over the input sequence are attached to every alert before it is forwarded to the SOC over Kafka. The deployment context is shown in Figure 1.

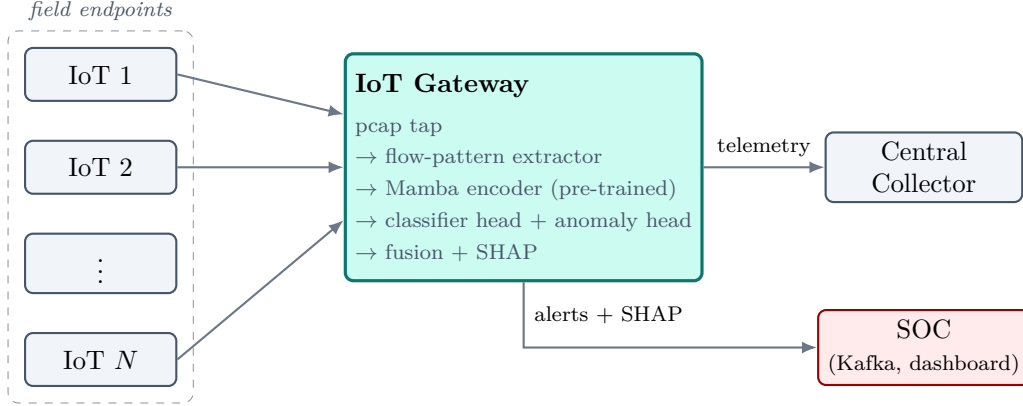


Figure 1: Deployment context. The detector is contained entirely within the IoT gateway, the natural choke point on the upstream path. Telemetry continues to the central collector unmodified; alerts and SHAP attributions are forwarded out-of-band to the SOC. The detector does not gate telemetry delivery.

3.2 Flow-pattern representation

For each bidirectional flow we extract its first K packets ($K \approx 32$, ablated). Each packet is reduced to a fixed-dimensional feature vector $p_t \in \mathbb{R}^d$ comprising:

- Packet length and direction (uplink vs. downlink).
- Inter-arrival time since the previous packet of this flow.
- TCP flag bits (SYN, ACK, FIN, RST, PSH, URG).
- Selected IP / TCP header-level fields (TTL, window size, header length).
- Optionally the first M bytes of the packet (per ET-BERT and NetMamba). These bytes are treated as an opaque pattern; the model never parses payload semantics, so the approach remains valid on encrypted traffic.

The flow is then the length- K sequence $\mathbf{x} = (p_1, p_2, \dots, p_K)$. This contrasts deliberately with the conventional aggregated representation, as shown in Figure 2: the aggregated vector is permutation-invariant and discards order, while the sequence preserves the temporal structure that the downstream sequence model can exploit.

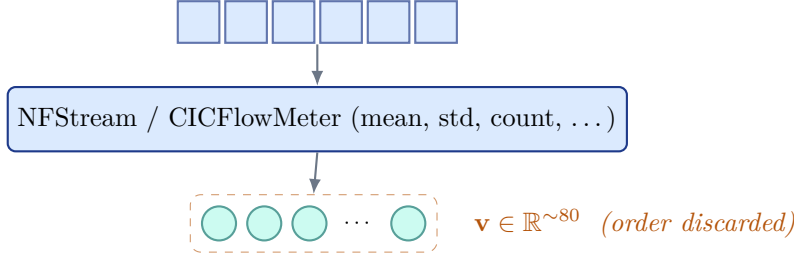
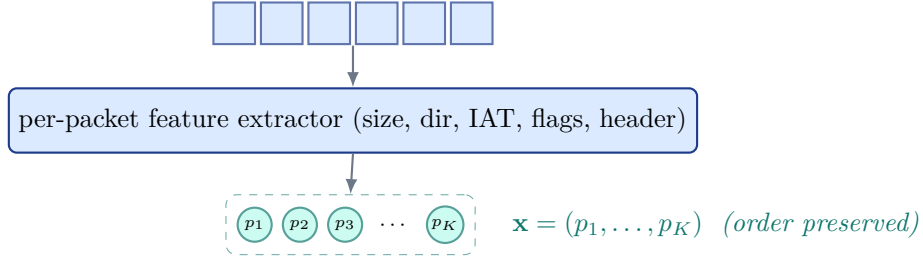
Aggregated representation (conventional)**Flow-pattern representation (ours)**

Figure 2: The two input representations. The conventional aggregated vector (top) collapses an entire flow into ≈ 80 summary statistics and is invariant to packet ordering — temporal patterns are erased before the model ever sees them. The flow-pattern representation (bottom) preserves the ordered per-packet features as a sequence, letting a sequence model learn temporal structure such as beaconing intervals, scan progressions, and burst behaviour. This is the input shift the proposal commits to.

3.3 The Mamba encoder

The encoder is a stack of Mamba blocks operating over the length- K sequence. We start from a small configuration (4–6 blocks, hidden dimension 128) and adjust based on gateway-hardware ablation. A learned positional embedding is added to the per-packet features before the first block. The encoder produces a sequence of hidden states $(h_1, \dots, h_K)$; we use the final hidden state $\mathbf{z} = h_K$ as the flow-level representation, with mean-pooling as an alternative. The encoder is the only component shared between supervised and anomaly tasks — by design, both heads operate on the same $\mathbf{z}$.

3.4 Downstream heads

Classification head. A two-layer MLP on top of $\mathbf{z}$, followed by softmax over the seven CICIoT2023 attack categories (DDoS, DoS, Recon, Web, Brute Force, Spoofing, Mirai). Trained with focal loss ($\gamma \approx 2$) to compensate for heavy class imbalance.

Anomaly head. A one-class scorer fitted on benign embeddings. Crucially, the anomaly head does *not* consume the encoder output $\mathbf{z}$ directly: we first apply a learnable **projection MLP** $g_\phi : \mathbf{z} \mapsto \mathbf{z}_{\text{ano}}$ that compresses to a substantially lower-dimensional anomaly space (e.g., $\mathbb{R}^{128} \rightarrow \mathbb{R}^{32}$). One-class methods like deep SVDD are sensitive to the curse of dimensionality — in high dimensions, distances concentrate and lose discriminative power — so projecting down sharpens the score. The projection is trained jointly with the deep SVDD objective on benign embeddings. We default to deep SVDD (a learned hypersphere around the benign-embedding

centroid) over $\mathbf{z}_{\text{ano}}$; Mahalanobis distance to the benign mean (also in the projected space) is the fallback. Threshold is calibrated on a held-out benign set at the 99th percentile (target 1% FPR).

Fusion. An alert is raised if $s_{\text{cls}} \geq \tau_{\text{cls}}$ for any attack class *or* $s_{\text{ano}} \geq \tau_{\text{ano}}$. The two thresholds are calibrated independently. A learned fusion is deferred to future work.

3.5 Inference path

Figure 3 shows a flow’s full path through the deployed detector. The path is single-pass: each flow is encoded once, both heads consume the same embedding (with a learned projection in front of the anomaly head), and only flagged flows pay the cost of SHAP attribution. SHAP is computed over the input sequence with DeepExplainer, producing per-packet-position attributions that an analyst can read directly (*e.g.*, “flagged because of unusual IAT pattern in packets 4–7”).

3.6 Two operating modes — default and strong

The gateway runs in one of two configurations, and the choice is an **operator-level setting** flipped by the admin, not a per-flow decision. One mode runs continuously by default; the other is switched on when the situation calls for it.

- **Default mode — low power, always on.** Smaller Mamba configuration, smaller K (*e.g.*, $K = 8$), only the anomaly head active, no SHAP. This is the baseline operating point: detection is on, the compute cost is small enough for continuous use on Raspberry-Pi-class hardware, and the system catches both known and novel attacks via the anomaly score alone. Target < 2 ms per flow.
- **Strong mode — admin-switched, full pipeline.** Full Mamba configuration, $K = 32$, both classifier and anomaly heads (the latter via the projection MLP), plus SHAP attribution on every alert. Activated by the admin when needed — elevated threat level, active incident investigation, high-value time window, or simply when the operator wants the maximum detection fidelity. Target < 10 ms per flow.

Switching is a configuration change applied to the gateway, not a per-flow decision and not an escalation. The same model artifacts — encoder θ_* , classifier head, projection, anomaly head — are used in both modes; what differs is which components are engaged and at what size. Both modes produce alerts that go to the SOC. Strong mode additionally produces per-packet SHAP attributions for every alert.

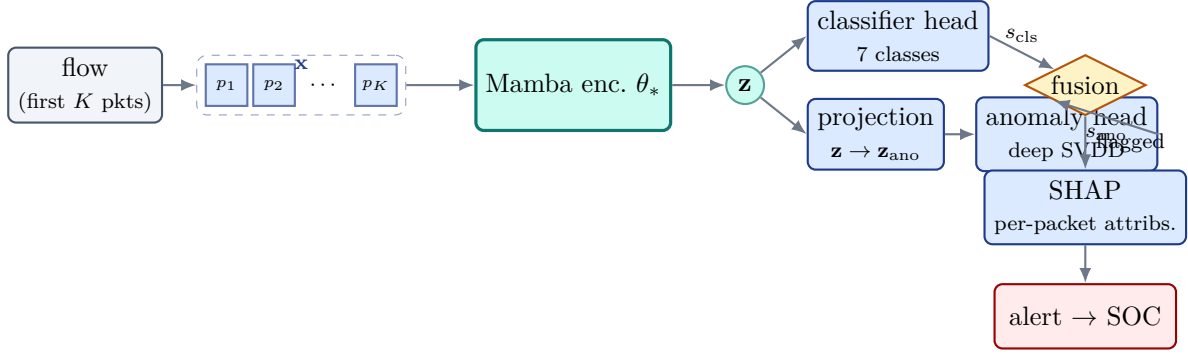


Figure 3: Inference pipeline. A flow is converted to its first- K -packet sequence $\mathbf{x}$, encoded once by the pre-trained Mamba encoder θ_* to the embedding $\mathbf{z}$, and read by two parallel heads. The classifier head consumes $\mathbf{z}$ directly. The anomaly head consumes a lower-dimensional projection $\mathbf{z}_{\text{ano}}$ (a learned MLP from $\mathbf{z}$), which mitigates the curse of dimensionality that one-class scorers like deep SVDD are sensitive to. A rule-based fusion gate raises an alert if either head crosses threshold; SHAP DeepExplainer then produces per-packet-position attributions over $\mathbf{x}$, and the alert is forwarded to the SOC.

4 Training Pipeline

Training runs in three stages, summarised in Figure 4. The encoder is built up first (without labels), then specialised for classification, and finally augmented with an anomaly head.

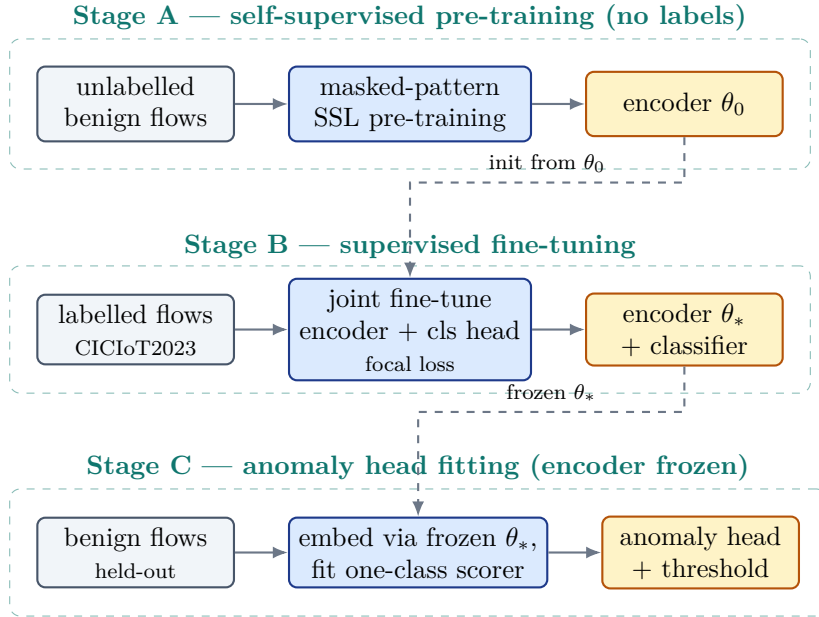


Figure 4: Training pipeline. Stage A pre-trains the encoder on unlabelled benign flows with a masked-pattern objective; no attack labels are used. Stage B initialises from θ_0 and jointly fine-tunes the encoder and a supervised classification head on labelled CICIoT2023 attack categories, yielding θ_* and the classifier. Stage C freezes θ_* , embeds a held-out benign set, fits the one-class anomaly scorer (deep SVDD by default) on those embeddings, and calibrates the threshold. At inference (Figure 3), the encoder θ_* is shared by both heads.

4.1 Stage A — Self-supervised pre-training

The Mamba encoder is pre-trained on benign-only CICIOT2023 flows. We use a masked-pattern objective: a random subset of packet positions in the input sequence is masked, and the encoder is trained to predict the masked feature vectors from their context. This is the analogue of masked language modelling, adapted to per-packet features. A contrastive variant over augmented flow views (per ET-SSL) is included as an alternative pre-training objective and ablated. No attack labels are used in this stage. If GPU budget proves the binding constraint, we warm-start from a publicly available Mamba checkpoint and adapt rather than train from scratch.

4.2 Stage B — Supervised fine-tuning

We initialise the encoder from θ_0 and add a two-layer MLP + softmax classification head. The encoder and head are jointly fine-tuned on the labelled CICIOT2023 training set using focal loss. We additionally train a baseline CNN-BiLSTM on aggregated flow statistics in the same stage, so the gain from the flow-pattern + Mamba combination can be cleanly attributed.

4.3 Stage C — Anomaly head fitting

We freeze the fine-tuned encoder θ_* , embed a held-out benign validation set, and fit a one-class scorer on those embeddings. Deep SVDD is the default; Mahalanobis is the fallback. The threshold is calibrated at the 99th percentile of benign validation scores. The held-out benign set is disjoint from the supervised training set.

4.4 Validation of zero-day capability

Because the anomaly head is trained without seeing any attack data, we cannot evaluate it by simple held-out test split. We instead evaluate it on *attack classes that were held out from supervised training* — a leave-one-class-out protocol. The anomaly head’s detection rate on these unseen classes is our zero-day proxy.

4.5 Progressive learning — absorbing new attack types

The three-stage training pipeline is also designed to make ongoing updates cheap. When a previously unseen attack type appears in the wild, the system handles it in four steps without retraining from scratch:

1. **Day-zero detection by the anomaly head.** The one-class scorer flags the novel attack as anomalous because it does not look like benign traffic. The analyst sees the alert and the accompanying SHAP attributions.
2. **Labelling.** The SOC confirms a sample of flagged flows as attack instances and assigns a new class label.
3. **Head-only fine-tuning.** The classifier head is extended with the new class (one additional softmax output) and re-fine-tuned on the new labelled data with a brief replay buffer of the original classes to prevent catastrophic forgetting. The Mamba encoder θ_* remains *frozen* during this update. This is a small training run — typically minutes on a single GPU — because only the lightweight classification head changes.
4. **Optional anomaly recalibration.** If the new attack distribution shifts the benign-vs-attack boundary in the projected anomaly space, the SVDD threshold is recalibrated on a fresh benign validation set. The encoder and projection stay fixed.

Full encoder retraining is only required if the *benign* traffic distribution itself shifts (new device classes deployed, network re-architecture, etc.) — a much rarer event than new-attack arrival. This decoupling of "what's normal" (encoder, expensive to retrain) from "what's malicious" (heads, cheap to update) is the operational payoff of the shared-encoder design.

5 Evaluation Plan

5.1 Datasets

CICIoT2023 (primary): supervised training, primary test, and benign pre-training corpus. 105 real devices, 33 attack types, 7 categories. **TON_IoT**: cross-dataset generalisation target, never seen during training. **IoT-23**: real-world malware capture (Mirai, Torii, Okiru, etc.), used as a malware-specific zero-day check.

5.2 Baselines and what each one quantifies

- **ET-BERT** (reference upper bound). Quantifies how much accuracy the Mamba detector retains relative to the transformer-class reference at gateway-prohibitive cost. Either run directly on a subset or use published numbers.
- **CNN-BiLSTM on aggregated statistics** (representation control). Isolates the gain from flow-pattern input + Mamba over the conventional aggregated-statistic + sequence-model pipeline.
- **XGBoost on aggregated statistics** (the classical-floor challenge). In the IDS literature, gradient-boosted trees on hand-crafted flow features are routinely competitive with — and sometimes beat — deep models on in-distribution test sets. We keep XGBoost as an explicit baseline precisely to answer the “do we need deep learning at all?” question. The deep model only earns its complexity if it outperforms XGBoost on the metrics XGBoost *cannot* target: zero-day detection (XGBoost is supervised-only), cross-dataset transfer (the pre-trained encoder is the bet for better transfer), and adversarial robustness. If XGBoost matches our detector on CICIoT2023 in-distribution alone, we report that as a finding, not a failure.
- **Isolation Forest** on aggregated statistics (classical unsupervised baseline). Establishes a non-deep zero-day floor against which the anomaly head must improve.

5.3 Metrics

Per-class precision, recall, F1; ROC-AUC; PR-AUC (the more honest metric under heavy class imbalance); inference latency at p50, p95, p99; model size in MB post-quantisation. Pre-training is monitored separately via masked-prediction accuracy and contrastive loss curves.

5.4 Adversarial robustness

FGSM perturbations on the per-packet feature sequence, constrained to remain in valid feature ranges (we do not allow physically impossible perturbations such as negative packet sizes). We sweep perturbation magnitude ε and report the detection-rate degradation curve for both heads. Following the recent IoT-IDS adversarial-robustness work [12], we also test whether SHAP attribution fingerprints reliably separate clean and adversarial inputs — a practical secondary defence.

5.5 Edge benchmark

After supervised training, the encoder and classifier are quantised to int8 (post-training quantisation, no fine-tuning required). We measure inference latency and peak memory on a Raspberry Pi 4 (target hardware to be confirmed with the mentor). Sub-10 ms p95 per-flow inference is the published edge target and our acceptance criterion.

6 Project Plan

6.1 Twelve-week timeline

Week	Milestone
1–2	Environment setup; download and validate CICIoT2023, TON_IoT, IoT-23; reproduce one published number on each as sanity check.
3	Flow-pattern extraction pipeline (pcap $\rightarrow$ per-packet sequence); classical baselines (Isolation Forest, XGBoost) on aggregated statistics.
4–5	Stage A — self-supervised pre-training of the Mamba encoder on benign CICIoT2023; ablate K , encoder depth, masked-fraction.
6	Stage B — supervised fine-tuning of encoder + classifier head; bring up CNN-BiLSTM baseline on aggregated statistics; bring up ET-BERT comparison.
7	Stage C — one-class anomaly head fitting; threshold calibration; held-out-class zero-day evaluation.
8	Cross-dataset evaluation on TON_IoT and IoT-23.
9	SHAP integration on flagged alerts; qualitative case studies on true- and false-positive examples.
10	FGSM adversarial study against both heads; SHAP-fingerprint adversarial-detection check.
11	Post-training int8 quantisation; latency / memory benchmark on gateway hardware.
12	One stretch extension (if time); final report; DRDO presentation.

6.2 Deliverables

1. Reproducible code repository covering the full pipeline (flow-pattern extraction, pre-training, fine-tuning, anomaly head, evaluation, SHAP, FGSM, quantisation).
2. Pre-trained Mamba encoder checkpoint θ_* released alongside the report.
3. Benchmark table: Mamba detector vs. ET-BERT vs. CNN-BiLSTM vs. Isolation Forest vs. XGBoost, across CICIoT2023, TON_IoT, IoT-23.
4. Adversarial-robustness study: detection rate vs. ε curves for both heads.
5. Latency and memory profile of the quantised model on gateway hardware, in tabular form.
6. Technical report (≈ 25 – 30 pages) and a DRDO-presented summary.

6.3 Risks and mitigations

- **Pre-training compute budget.** A from-scratch Mamba pre-training run may exceed available GPU time. *Mitigation:* small encoder configuration; restricted pre-training corpus; warm-start from a public Mamba checkpoint if needed.
- **Class imbalance hiding minority-class failure.** *Mitigation:* focal loss; always report per-class precision/recall/F1 alongside aggregate metrics.

- **Cross-dataset degradation**, the standard caveat in IDS literature. *Mitigation*: report honestly, measure rather than target; do not over-claim.
- **Gateway memory overrun**. *Mitigation*: post-training int8 quantisation; fallback to a smaller Mamba configuration; reduce K as last resort.
- **Byte-level adversarial sensitivity**. Byte-level input is known to be more perturbation-sensitive than aggregated statistics. *Mitigation*: the FGSM study is designed to surface and quantify this, not hide it; SHAP-fingerprint defence is the secondary signal.
- **Label noise in CICIOT2023**, documented in follow-up analyses. *Mitigation*: spot-check a sample of training examples manually; report any patterns we find.

6.4 Stretch goals

1. **Mamba-2 / SSD backbone swap**. If Mamba-1 leaves latency / memory headroom at the gateway, upgrade to Mamba-2 for higher-quality representations at comparable cost.
2. **Graph-aware branch (E-GraphSAGE)**. Add a third head operating on a per-window device-communication graph; static-per-window graph keeps the branch within edge budget.
3. **Federated pre-training**. Distributed pre-training across multiple DRDO sensor sites that cannot share raw pcaps, using FedAvg-style aggregation of the masked-pattern objective.

References

References

- [1] Lin, X., Xiong, G., Gou, G., Li, Z., Shi, J., Yu, J. (2022). ET-BERT: A Contextualized Datagram Representation with Pre-training Transformers for Encrypted Traffic Classification. *Proc. WWW '22*. arxiv.org/abs/2202.06335
- [2] Wang, T., *et al.* (2024). NetMamba: Efficient Network Traffic Classification via Pre-training Unidirectional Mamba. *arXiv:2405.11449*. arxiv.org/abs/2405.11449
- [3] Anomaly Detection in Encrypted Network Traffic Using Self-Supervised Learning (2025). *Scientific Reports*, Nature. nature.com/articles/s41598-025-08568-0
- [4] Gu, A., Goel, K., Ré, C. (2022). On the Parameterization and Initialization of Diagonal State Space Models. *NeurIPS*. arxiv.org/abs/2206.11893
- [5] Gu, A., Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *arXiv:2312.00752*. arxiv.org/abs/2312.00752
- [6] Dao, T., Gu, A. (2024). Transformers are SSMS: Generalized Models and Efficient Algorithms Through Structured State-Space Duality. *ICML*. arxiv.org/abs/2405.21060
- [7] Neto, E.C.P., *et al.* (2023). CICIOT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment. *Sensors*, 23(13):5941, MDPI. mdpi.com/1424-8220/23/13/5941
- [8] Morshedi, M., *et al.* (2025). A Comprehensive Review of Deep Learning Techniques for Anomaly Detection in IoT Networks. *Engineering Reports*, Wiley. onlinelibrary.wiley.com/doi/full/10.1002/eng2.70415
- [9] A Survey of AI-Based Anomaly Detection in IoT and Sensor Networks (2023). *Sensors*, 23(3):1352, MDPI. mdpi.com/1424-8220/23/3/1352

- [10] Evaluating Large Transformer Models for Anomaly Detection of Resource-Constrained IoT Devices (2025). *Scientific Reports*, Nature. [nature.com/articles/s41598-025-21826-5](https://www.nature.com/articles/s41598-025-21826-5)
- [11] Explainable AI for Zero-Day Attack Detection in IoT Networks (2025). *Discover Internet of Things*, Springer. [doi:10.1007/s43926-025-00184-8](https://doi.org/10.1007/s43926-025-00184-8)
- [12] Enhancing Adversarial Robustness of IoT IDS (2025). *arXiv:2511.06197*. arxiv.org/abs/2511.06197
- [13] Lo, W.W., *et al.* E-GraphSAGE: A Graph Neural Network Based Intrusion Detection System for IoT. *arXiv:2103.16329*. arxiv.org/abs/2103.16329
- [14] Hybrid DL–Federated Learning Powered IDS for IoT/5G Edge (2025). *arXiv:2509.15555*. arxiv.org/abs/2509.15555
- [15] Federated Learning for Intrusion Detection in IoT Environments (2025). *Discover Internet of Things*, Springer. [doi:10.1007/s43926-025-00169-7](https://doi.org/10.1007/s43926-025-00169-7)